

Système de stockage distribué basé sur une DHT

Projet Master Systèmes et Applications Répartis (SAR)

PSAR – Sorbonne Université

Auteurs: Cedric Genevieve, Safa Gunes, Somaya Boudalil
Étudiants en Master 2026 — Systèmes et Applications Répartis

Instructeur: Prof. Alessio Pagliari

Date: 15 février 2026

Table des matières

1	Introduction aux DHTs : Résumé des notes de cours	2
1.1	Introduction au problème du Lookup (ref. Section 7.1)	2
1.2	Comparaison des stratégies de recherche (ref. Section 7.1.1 - 7.1.5)	2
1.3	Fondamentaux des Distributed Hash Tables (ref. Section 7.2)	3
1.4	Mécanismes des DHTs (ref. Section 7.3)	4
1.5	Interfaces des DHTs (ref. Section 7.4)	5
1.6	Conclusions (ref. Section 7.5)	6
2	Chord	7
2.1	Design du système et philosophie	7
2.2	Consistent Hashing et l'Identifier Circle	7
2.3	Localisation de clés scalable : Finger Tables	7
2.4	Théorèmes	8
2.5	Algorithmes des nœuds	9
2.6	Algorithmes de la Finger Table	10

1 Introduction aux DHTs : Résumé des notes de cours

Je me suis largement inspiré de l'intégralité de ces notes pour fournir une vue d'ensemble complète du sujet. J'ai trouvé ces notes de cours particulièrement exhaustives car elles couvrent, à mon sens, tous les aspects : de l'histoire à l'adressage de bas niveau et au *"content-based routing"*, jusqu'aux interfaces applicatives de haut niveau. Ce document offre une feuille de route complète pour comprendre comment les systèmes modernes massivement distribués restent stables sous une *"heavy network load"*.

Référence : DBLP:conf/p2p/WehrleGR05.

1.1 Introduction au problème du Lookup (ref. Section 7.1)

Le document débute en notant une transition des systèmes distribués vers des réseaux massifs comptant des millions de participants, tels que Skype et BitTorrent. Ces systèmes doivent gérer la *"scalability, flexibility, and instant deployment"* sans aucun contrôle centralisé.

Au cœur de ces réseaux se trouve le **lookup problem**. Je trouve intéressant la manière dont le document indique que ce problème peut être réduit à une seule question :

"Où stocker, et comment trouver un certain élément de donnée D dans un système distribué sans aucun contrôle ou coordination centralisés ?".

L'objectif est qu'un nœud puisse récupérer un élément de donnée D spécifique sans avoir de connaissance préalable de l'emplacement actuel de D.

1.2 Comparaison des stratégies de recherche (ref. Section 7.1.1 - 7.1.5)

Pour justifier l'usage des DHTs, le texte compare trois méthodes principales de gestion et de récupération des données :

1. Central Server

Cette approche de "première génération" (ex: Napster) implique un serveur central qui garde trace de l'emplacement de chaque élément.

En termes de performance, elle est très rapide, avec une complexité de recherche de $O(1)$. De plus, elle permet des *"fuzzy and complex queries"* car le serveur possède une vue globale.

Cependant, elle représente un *"single point of failure"*. Si le serveur échoue ou est attaqué, l'ensemble du système devient inutilisable.

2. Flooding Search

Utilisée par les systèmes de "deuxième génération" comme Gnutella, cette méthode supprime totalement le serveur.

Le mécanisme consiste en l'envoi par un nœud d'une requête à ses voisins, qui la propagent par *"flood"* aux nœuds suivants jusqu'à ce qu'un *"Time to Live"* (TTL) soit atteint.

Sa scalabilité est médiocre car l'*"communication overhead"* est extrêmement élevé. De plus, les résultats de recherche ne sont pas garantis car la recherche ne couvre qu'un nombre limité de *hops*, n'offrant donc pas la fiabilité requise pour ces applications.

3. Distributed Hash Tables (DHTs)

Le document décrit les DHTs comme la *"golden path"* évitant les inconvénients des deux autres méthodes.

Elles sont souvent appelées *"structured Peer-to-Peer systems"* car elles utilisent des procédures proactives pour organiser le réseau. Chaque nœud ne gère qu'un petit nombre de références ($O(\log N)$), et les données peuvent être localisées en $O(\log N)$ hops.

Réf : La Table 7.1 et la Figure 7.4 fournissent une comparaison directe de ces complexités, soulignant que les DHTs offrent le meilleur équilibre entre efficacité et robustesse.

1.3 Fondamentaux des Distributed Hash Tables (ref. Section 7.2)

Cette section explore la stratégie proactive utilisée par les DHTs pour gérer les données, garantissant que chaque nœud possède un rôle spécifique et prévisible dans le réseau. Les DHTs gèrent les données en les distribuant entre les nœuds et en utilisant un *"deterministic routing scheme"*.

Gestion distribuée des données (ref. Section 7.2.1)

Contrairement aux systèmes non structurés qui reposent sur un *"reactive flooding"* pour trouver des éléments, une DHT est *"proactive"*.

Chaque nœud se voit assigner une *"range of data items"* spécifique dont il est responsable. Chaque nœud maintient une *"partial view"* de l'ensemble du système, stockant un nombre limité de liens vers d'autres nœuds.

Le processus de routage traverse plusieurs nœuds, se rapprochant de la destination à chaque hop jusqu'à ce que la cible soit atteinte.

Adressage dans les Distributed Hash Tables (ref. Section 7.2.2)

Les DHTs fonctionnent en créant un *"address space"* virtuel qui existe indépendamment de la topologie physique du réseau.

Ces espaces consistent généralement en de très grands entiers (par exemple de 0 à $2^{160} - 1$). Chaque nœud participant se voit attribuer une *"contiguous portion"* de cet espace d'adressage.

Les éléments de données reçoivent un *"identifier ID"* unique dans le même espace, généralement en utilisant une *"collision-resistant hash function"* comme SHA-1 sur le nom du fichier ou son contenu.

Comme illustré dans la Figure 7.6, l'espace d'adressage est souvent interprété géométriquement comme un cercle ou un anneau.

Routage (ref. Section 7.2.3)

Le routage est la fonctionnalité centrale qui résout le *lookup problem*. Les nœuds ne connaissent pas tous les autres nœuds ; ils ne stockent qu'un *"bounded number of links"*.

Si un nœud reçoit un message pour un ID qu'il ne possède pas, il le transmet au nœud de sa table qui est numériquement le plus proche de l'ID cible. Ce processus récursif est conçu pour trouver efficacement la destination en un *"small number of hops"*.

Méthodes de stockage (ref. Section 7.2.4)

Il existe deux manières principales de stocker les données une fois le nœud responsable localisé, comme illustré dans la Figure 7.8 :

Direct Storage : La donnée réelle est copiée sur le nœud responsable. C'est excellent pour la *availability* mais consomme plus de bande passante et de stockage.

Indirect Storage : Le nœud ne stocke qu'un "*pointer*" ou une référence vers l'endroit où la donnée est réellement conservée. Cela réduit la charge, mais signifie que la donnée est perdue si le nœud d'origine se déconnecte.

1.4 Mécanismes des DHTs (ref. Section 7.3)

Cette section se concentre sur la gestion des données et des nœuds, spécifiquement sur la manière dont le système gère l'arrivée, le départ et l'occurrence fréquente de *node failure*.

Avant qu'un nœud puisse interagir avec les données, il doit rejoindre le réseau. Cette arrivée déclenche des changements dans l'infrastructure, nécessitant la mise à jour des informations de routage et de la distribution des données. À l'inverse, le système doit détecter et s'adapter chaque fois qu'un nœud quitte le réseau ou échoue.

Arrivée de nœuds (ref. Section 7.3.2)

Le document spécifie un processus en quatre étapes pour qu'un nouveau nœud rejoigne la DHT :

1. **Bootstrap** : Le nouveau nœud utilise une méthode de *bootstrap* pour contacter un nœud existant arbitraire, qui sert d'"*entry point*".
2. **Address Assignment** : Le nœud reçoit sa propre "*partition in the logical address space*".
3. **Routing Update** : Les autres nœuds du système mettent à jour leurs informations de routage pour reconnaître le nouveau membre.
4. **Data Transfer** : Le nouveau nœud récupère les paires (*key, value*) dont il est désormais responsable auprès du nœud qui les détenait précédemment.

Défaillance de nœuds (ref. Section 7.3.3)

Parce que les systèmes distribués consistent souvent en des machines de bureau peu fiables et mal connectées, les défaillances sont fréquentes. Les DHTs gèrent cela via deux types de récupération :

Reactive Recovery : Si un nœud rencontre un pair défaillant lors d'une opération régulière, il gère la faute en trouvant des routes alternatives.

Proactive Recovery : Le système sonde périodiquement les autres nœuds ("*probes*") pour vérifier leur statut. Si un nœud est non opérationnel, son entrée de routage est remplacée.

En cas d'échec d'un nœud, les données applicatives qu'il stockait sont perdues, à moins que la "*replication*" ne soit utilisée. Certains systèmes utilisent une approche "*soft-state*" où les données sont supprimées ("*pruned*") à moins que l'application ne les rafraîchisse périodiquement.

Départ de nœuds (ref. Section 7.3.4)

Bien que les nœuds puissent être traités comme des "*failed nodes*" s'ils disparaissent, les DHTs préfèrent que les nœuds notifient le système avant de partir. Ce départ volontaire permet au système

de copier immédiatement les données du nœud sortant vers son *successor*, de mettre à jour le routage plus rapidement (améliorant l’*“routing efficiency”*) et de garantir le bon fonctionnement des mécanismes de *“replication and load-balancing”*.

1.5 Interfaces des DHTs (ref. Section 7.4)

Les notes de cours mentionnent deux angles principaux pour considérer la fonctionnalité d’une DHT : comme un système de routage ou comme un système de stockage.

Interface de routage (ref. Section 7.4.1)

Cette *“stateless interface”* se concentre sur la livraison de paquets basés sur un ID de destination plutôt que sur une adresse IP physique.

La primitive *send* accepte un ID de destination et un message, le livrant au nœud responsable depuis n’importe quel point du réseau. La primitive *receive* transmet les messages entrants et leurs identifiants à l’application sur le nœud récepteur. Cette interface étant générique, les développeurs peuvent implémenter leurs propres stratégies de stockage et de *load-balancing*.

Interface de stockage (ref. Section 7.4.2)

Cette couche ajoute de la complexité en gérant la persistance et la fiabilité des données. Comme le montre la Figure 7.9, elle imite une table de *hash* standard.

put prend une paire (*key, value*) et la stocke sur le nœud responsable. *get* accepte un identifiant et renvoie la valeur associée. La couche de stockage doit gérer les *“routing failures”*, prévenir la perte de données via la *“replication”* et gérer l’*“admission control”*.

Interface Client (ref. Section 7.4.3)

Le document note également que des nœuds peuvent agir comme de simples *“clients”*. Cela permet à des hôtes non-participants d’utiliser les services de la DHT sans rejoindre le réseau *overlay*, ce qui est utile pour la fiabilité ou l’*“access control”*.

1.6 Conclusions (ref. Section 7.5)

Le texte conclut en pesant les avantages significatifs des DHTs face à leurs défis persistants. Bien qu'elles résolvent les problèmes de *scalability* des systèmes centralisés, elles ne constituent pas une solution universelle.

Défis de conception

Les développeurs doivent naviguer entre plusieurs objectifs conflictuels :

- **Routing Efficiency** : Influencée par le nombre de références aux autres nœuds et la connaissance de la topologie au niveau IP.
- **Management Overhead** : Le coût de fond pour maintenir la DHT même en l'absence de trafic actif.
- **Churn** : Les taux élevés de *joins* et *leaves* simultanés exercent un stress particulier sur la stabilité globale, pouvant mener à des systèmes défectueux.

Limitations fondamentales

Malgré leur efficacité, les DHTs font face à des défis liés au principe de l'indexation distribuée :

- **Untrusted Environments** : Difficulté d'opérer de manière fiable lorsque les nœuds sont malveillants (*Byzantine faults*).
- **Query Constraints** : La récupération étant basée sur des identifiants numériques, effectuer des "*fuzzy searches*" ou des requêtes textuelles complexes est extrêmement difficile à réaliser efficacement.

2 Chord

Chord est un protocole de *lookup* distribué qui répond à un problème fondamental : localiser efficacement le nœud stockant une donnée particulière. Alors que de nombreux systèmes *peer-to-peer* offrent des fonctionnalités complexes comme l'anonymat ou le stockage redondant, Chord se concentre exclusivement sur une opération centrale : le *mapping* d'une *key* vers un nœud.

2.1 Design du système et philosophie

Chord est conçu comme une bibliothèque liée aux applications clientes et serveurs. Il répond à cinq défis critiques des systèmes *peer-to-peer* (P2P) :

1. **Load Balance** : Chord agit comme une fonction de *hash* distribuée, répartissant les *keys* uniformément pour éviter qu'un seul nœud ne soit submergé.
2. **Decentralisation** : Aucun nœud n'est plus important qu'un autre, ce qui élimine les *single points of failure* tels que ceux rencontrés dans Napster.
3. **Scalability** : Le coût du *lookup* croît selon le logarithme du nombre de nœuds, rendant possible l'exploitation de systèmes impliquant des millions de participants.
4. **Availability** : Le système s'ajuste automatiquement aux *joins* et aux *failures*, garantissant que le nœud responsable soit toujours localisé.
5. **Flexible Naming** : Chord utilise un *flat key space*, offrant aux applications une liberté totale quant à la manière de lier leurs noms internes aux *keys* de Chord.

2.2 Consistent Hashing et l'Identifiant Circle

Chord utilise une variante du *consistent hashing* pour assigner les *keys* aux nœuds.

Les nœuds ainsi que les *keys* reçoivent un **identifiant** de m bits via une fonction de *hash* de base telle que SHA-1. L'ID d'un nœud est dérivé de son adresse IP, tandis que l'ID d'une *key* est le *hash* de la donnée elle-même.

Les identifiants sont disposés sur un cercle modulo 2^m appelé **chord ring**. Une *key* k est assignée au premier nœud dont l'identifiant est égal ou supérieur à k dans le sens des aiguilles d'une montre. Ce nœud est appelé le *successor* de la *key* k .

2.3 Localisation de clés scalable : Finger Tables

Pour éviter l'inefficacité d'un parcours linéaire du cercle, Chord maintient une table de routage appelée **finger table**.

Chaque nœud n maintient jusqu'à m entrées. La $i^{\text{ème}}$ entrée pointe vers le premier nœud qui succède à n d'au moins 2^{i-1} .

Approche par Binary Search : Cette structure permet à un nœud de stocker davantage d'informations sur les nœuds qui le suivent de près et moins sur ceux qui sont plus éloignés.

Performance : Chaque *lookup* réduit de moitié la distance vers la cible, garantissant que la destination est atteinte en $O(\log N)$ *hops*.

Les nœuds proches d'une *key* disposent d'informations de type "*fine-grained*", tandis que les nœuds plus éloignés permettent des "*long jumps*" à travers le réseau pour accélérer la recherche.

2.4 Théorèmes

L'article présente 8 théorèmes servant à justifier l'usage du *consistent hashing* et de la structure de la **finger table**.

Théorème 1 : *Pour tout ensemble de N nœuds et K keys, avec une probabilité élevée : chaque nœud est responsable d'au plus $(1 + \epsilon)K/N$ keys. Lorsqu'un $(N + 1)^{\text{ème}}$ nœud rejoint ou quitte le réseau, seules $O(K/N)$ keys sont déplacées.*

Ce premier théorème traite de l'efficacité avec laquelle Chord répartit les données sur le réseau, réalisant un "*natural load balance*". Il garantit qu'aucune machine ne devienne un goulot d'étranglement.

Théorème 2 : *Avec une probabilité élevée, le nombre de nœuds devant être contactés pour trouver un successor dans un réseau de N nœuds est en $O(\log N)$.*

Ce théorème démontre l'efficacité des *lookups*, même dans des systèmes comptant des millions de nœuds. Il justifie la structure de la **finger table**, où chaque *hop* est conçu pour réduire de moitié la distance vers l'ID cible.

Théorème 3 : *Avec une probabilité élevée, tout nœud rejoignant ou quittant un réseau Chord de N nœuds utilisera $O(\log^2 N)$ messages pour rétablir les finger tables et les invariants de routage.*

Le troisième théorème quantifie le coût du maintien de la structure du réseau lors de l'arrivée ou du départ de participants. Il permet aux développeurs d'estimer le "*management overhead*" du système. Il montre que bien qu'un *join* soit plus coûteux qu'un simple *lookup*, il reste très performant à mesure que le réseau croît.

Théorème 4 : *Dès qu'un nœud peut résoudre avec succès une requête donnée, il sera toujours en mesure de le faire à l'avenir.*

Cette propriété garantit qu'une fois que le réseau a atteint un état où un chemin vers une *key* existe, les routines de maintenance du protocole ne briseront pas cette connectivité.

Théorème 5 : *Passé un certain délai après le dernier join, tous les successor pointers seront corrects.*

Le protocole *stabilize* garantit la convergence, ramenant ainsi le réseau vers un état de cercle parfait une fois que le "*churn*" des nœuds entrants s'arrête.

Théorème 6 : *Si nous prenons un réseau stable de N nœuds, et qu'un autre ensemble allant jusqu'à N nœuds rejoint le réseau sans finger pointers (mais avec des successor pointers corrects), alors les lookups prendront toujours un temps en $O(\log N)$ avec une probabilité élevée.*

Cela prouve que Chord est extrêmement résilient lors de phases de croissance. Même si un nombre massif de nouveaux nœuds rejoint le réseau sans avoir encore construit leurs **finger tables**, l'infrastructure existante garantit que la vitesse de recherche ne se dégrade pas.

Théorème 7 : *Si nous utilisons une successor list de longueur $r = O(\log N)$ dans un réseau initialement stable, et que chaque nœud échoue avec une probabilité de $1/2$, alors avec une probabilité élevée *find_successor* renvoie le successor vivant le plus proche de la key demandée.*

Ce théorème valide le mécanisme de **Successor List**. Il prouve qu'avec seulement une quantité logarithmique de redondance, le système peut localiser les données de manière fiable, même si la moitié des machines du réseau disparaissent simultanément.

Théorème 8 : *Si nous utilisons une successor list de longueur $r = O(\log N)$ dans un réseau initialement stable, et que chaque nœud échoue avec une probabilité de $1/2$, alors le temps attendu pour exécuter `find_successor` dans le réseau défaillant est en $O(\log N)$.*

Il s'agit du pendant de performance du Théorème 7. Il garantit que non seulement la recherche trouvera le nœud correct après une défaillance de 50%, mais qu'elle le fera avec la même efficacité logarithmique.

2.5 Algorithmes des nœuds

1. L'algorithme de Successor Resolution

Le cœur de l'efficacité de Chord réside dans la fonction `find_successor`. Elle utilise la **finger table** pour effectuer des bonds exponentiels vers l'ID cible.

Algorithm: `Node.find_successor(id)`

```

1: if  $id \in (\text{node}, \text{successor}]$  then
2:   return successor
3: else
4:    $n' = \text{closest\_preceding\_node}(id)$ 
5:   return  $n'.\text{find\_successor}(id)$ 
6: end if
```

Pour trouver le nœud précédent le plus proche, le système itère à travers la **finger table**, du saut le plus grand vers le plus petit :

Algorithm: `Node.closest_preceding_node(id)`

```

1: for  $i = m$  downto 1 do
2:   if  $\text{finger}[i].\text{node} \in (\text{node}, id)$  then
3:     return  $\text{finger}[i].\text{node}$ 
4:   end if
5: end for
6: return node
```

2. Maintenance dynamique du ring : Joining et Stabilisation

Lorsqu'un nœud rejoint le réseau, il n'a besoin de connaître qu'un seul nœud existant pour s'intégrer au cercle. La fonction `stabilize` est exécutée périodiquement pour vérifier le *successor* et l'informer de l'existence du nœud.

Algorithm: `Node.join( $n'$ )`

```

1: predecessor = nil
2: successor =  $n'.\text{find\_successor}(\text{node})$ 
```

Algorithm: `Node.stabilize()`

```

1:  $x = \text{successor.predecessor}$ 
2: if  $x \in (\text{node}, \text{successor})$  then
3:   successor =  $x$ 
4: end if
5: successor.notify(node)
```

3. Résilience : Le mécanisme de Successor List

Pour gérer le "churn" des nœuds tombant en panne sans préavis, chaque nœud maintient une liste de r *successors*. Cela garantit que si le *successor* primaire échoue, le nœud peut toujours router vers le prochain nœud actif disponible dans la liste.

Algorithm: Node.check_predecessor()

```
1: if (predecessor has failed) then
2:   predecessor = nil
3: end if
```

Algorithm: Node.handle_successor_failure()

```
1: for each  $s \in \text{successor\_list}$  do
2:   if  $s$  is alive then
3:     successor =  $s$ 
4:     break
5:   end if
6: end for
```

2.6 Algorithmes de la Finger Table

1. Calcul des entrées de la Finger Table

Chaque nœud n maintient une table de routage de m entrées. La $i^{\text{ème}}$ entrée est définie comme le premier nœud qui succède au point $n + 2^{i-1}$ sur le cercle d'identifiants.

$$\text{finger}[i].\text{start} = (n + 2^{i-1}) \pmod{2^m}, \quad 1 \leq i \leq m$$

$$\text{finger}[i].\text{node} = \text{find_successor}(\text{finger}[i].\text{start})$$

2. Initialisation de la Finger Table

Lorsqu'un nouveau nœud rejoint le réseau, il doit renseigner sa **finger table**. Alors qu'une approche naïve consisterait à appeler `find_successor` m fois, le protocole utilise souvent des nœuds existants pour fournir des "hints", réduisant ainsi la complexité de l'initialisation.

Algorithm: Node.init_finger_table(n')

```

1: finger[1].node =  $n'$ .find_successor(finger[1].start)
2: predecessor = successor.predecessor
3: successor.predecessor = node
4: for  $i = 1$  to  $m - 1$  do
5:   if finger[ $i + 1$ ].start  $\in$  [node, finger[ $i$ ].node) then
6:     finger[ $i + 1$ ].node = finger[ $i$ ].node
7:   else
8:     finger[ $i + 1$ ].node =  $n'$ .find_successor(finger[ $i + 1$ ].start)
9:   end if
10: end for

```

3. Maintenance proactive de la table

Le réseau étant dynamique, les entrées des doigts peuvent devenir "*stale*" lors de l'arrivée ou du départ de nœuds. La routine `fix_fingers` est exécutée périodiquement pour rafraîchir une entrée aléatoire, garantissant que les informations de routage demeurent *eventually consistent*.

Algorithm: Node.fix_fingers()

```

1:  $i = \text{random index} \in [1, m]$ 
2: finger[ $i$ ].node = find_successor(finger[ $i$ ].start)

```

4. Mise à jour des autres nœuds

Lorsqu'un nouveau nœud n rejoint le réseau, il peut devenir le $i^{\text{ème}}$ doigt de plusieurs nœuds existants. La fonction `update_others` identifie ces nœuds et les force à mettre à jour leurs tables pour refléter l'arrivée de n .

Algorithm: Node.update_others()

```

1: for  $i = 1$  to  $m$  do
2:    $p = \text{find\_predecessor}(\text{node} - 2^{i-1})$ 
3:    $p.\text{update\_finger\_table}(\text{node}, i)$ 
4: end for

```

Algorithm: p.update_finger_table(s, i)

```

1: if  $s \in [\text{node}, \text{finger}[i].\text{node}]$  then
2:   finger[ $i$ ].node =  $s$ 
3:    $p = \text{predecessor}$ 
4:    $p.\text{update\_finger\_table}(s, i)$ 
5: end if

```